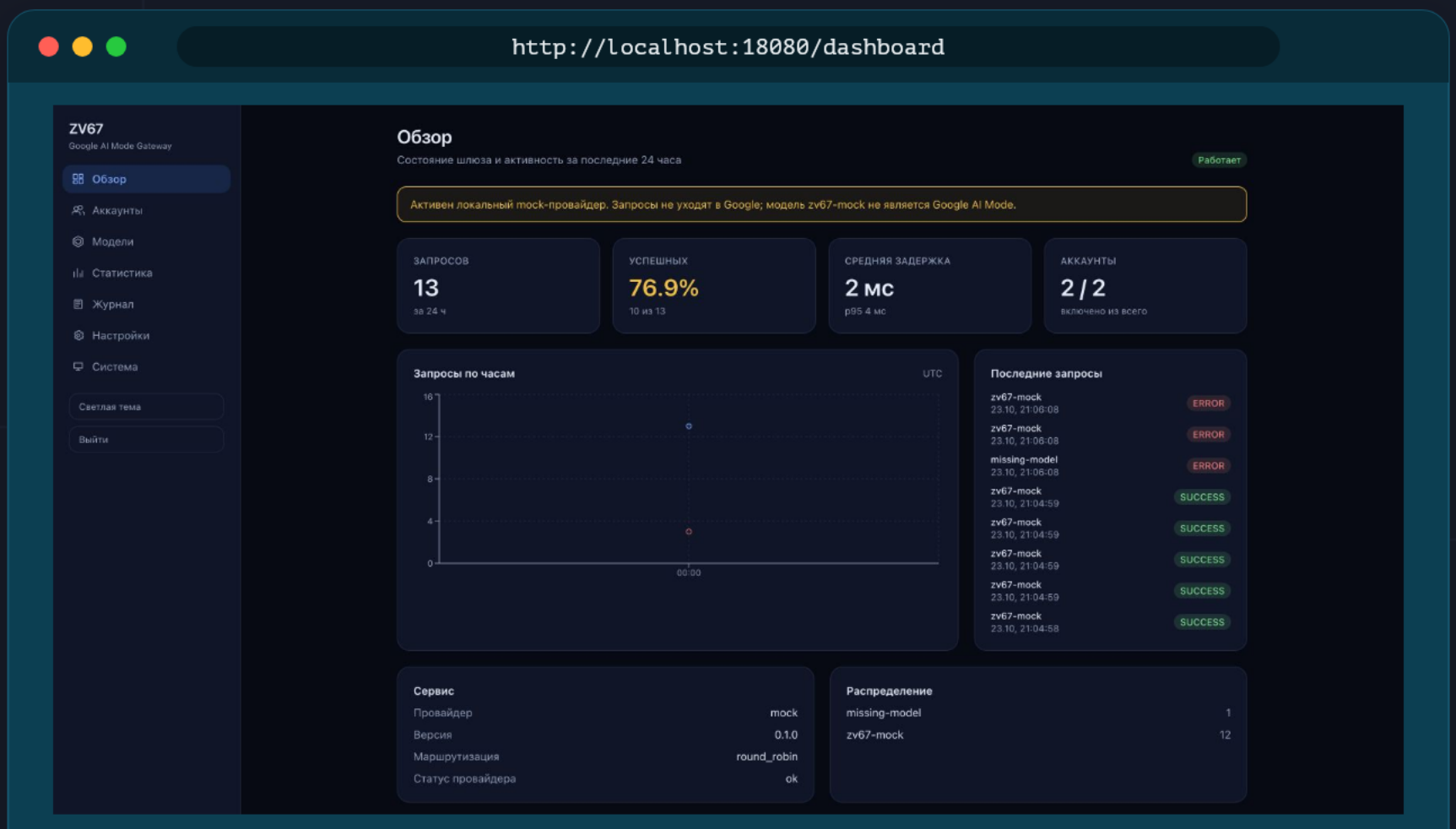
ШЛЮЗ АГЕНТА CODEX

ZV67: GOOGLE AI MODE GATEWAY

Бесшовный доступ к Google AI Mode из Codex без API-ключей



● ШЛЮЗ АКТИВЕН • ХОСТ LOCALHOST:18080 • СЕССИЯ ГОТОВА

API :18080

OPENAI API

- Стандартный эндпоинт completions
- Потоковая передача через SSE
- Прозрачный роутер OpenCodeX
- Нулевые задержки вызова

HEADLESS CDP

БЕЗ ОКОН БРАУЗЕРА

- Chrome в режиме --headless=new
- Управление через сокеты CDP
- Фокус окон и мыши не перебивается
- Фоновое исполнение в macOS

MULTI-ACCOUNT

МУЛЬТИАККАУНТЫ

- Изоляция профилей Chrome
- Раздельные cookies и cache
- Балансировка и ротация
- Обход персональных квот

FERNET 0600

ШИФРОВАНИЕ КУК

- Симметричный шифр Fernet (AES)
- Права 0600 на ключ в ~/.zv67/
- Куки расшифровываются в RAM
- Полное исключение из Git

ПРИВАТНЫЙ РЕПОЗИТОРИЙ • РАСПРОСТРАНЕНИЕ СТРОГО ПО ПРИГЛАШЕНИЮ

АРХИТЕКТУРА И ДАТАФЛОУ

ПОЛНЫЙ ПАЙПЛАЙН ОБРАБОТКИ ЗАПРОСА

Сквозной маршрут от вызова модели в Codex до потокового ответа

ШАГ 1 • КЛИЕНТ

CODEX DESKTOP / CLI

Пользователь выбирает модель zv67/*. Клиент отправляет стандартный JSON-запрос нативного формата chat/completions с системным промптом и списком tools.

ШАГ 2 • РОУТЕР

OPENCODEX PROXY

Распознаёт префикс zv67/ и перенаправляет трафик на локальный эндпоинт http://localhost:18080/v1 без изменения структуры сообщений диалога.

ШАГ 3 • ЯДРО ШЛЮЗА (КЛЮЧЕВОЙ УЗЕЛ)

ZV67 GATEWAY (FASTAPI :18080)

Перехватывает и нейтрализует tools (предотвращая ошибку 400 у веб-моделей). Выбирает активный Google-аккаунт, расшифровывает куки в RAM и управляет пулом. Логирует тайминги, метрики и ведёт учёт использования в базе SQLite.

ШАГ 4 • БРАУЗЕР

CHROME CDP SESSION MANAGER

Управляет процессом Google Chrome с флагом --headless=new через сокеты CDP. Изолирует профиль в отдельной папке и эмулирует действия авторизованного юзера.

ШАГ 5 • ИСТОЧНИК

GOOGLE AI MODE WEB (SSE)

Выполняет веб-поиск и синтез ответа в интерфейсе Google AI Mode. Транслирует Server-Sent Events обратно в шлюз, который стримит токены в Codex.

ПРЕИМУЩЕСТВА СВЯЗКИ ДЛЯ РАЗРАБОТЧИКА:

ПРОЗРАЧНОСТЬ ДЛЯ CODEX

Для Codex шлюз выглядит как официальный OpenAI-провайдер с поддержкой стриминга.

ЗАЩИТА ОТ БЛОКИРОВОК

Google видит естественные сетевые запросы живого браузера без признаков бота.

ЛОКАЛЬНЫЙ КОНТРОЛЬ

Все сокеты и сессии крутятся на вашей машине localhost без внешних посредников.

АВТОНОМНОСТЬ

Фоновый демон launchd гарантирует доступность сервиса при каждом старте системы.

БЕЗОПАСНОСТЬ И УЧЁТНЫЕ ДАННЫЕ

ИЗОЛЯЦИЯ И ЗАЩИТА GOOGLE-АККАУНТОВ

Гибкий импорт кук, симметричное шифрование и раздельные профили Chrome

УНИВЕРСАЛЬНЫЙ ИМПОРТ

ПОДДЕРЖКА ВСЕХ ФОРМАТОВ КУК

- **Cookie-Editor JSON**
  Нативный экспорт массива объектов [{"domain": ".google.com", "name": "SID", ...}]
- **Netscape cookies.txt**
  Стандартный табулированный текстовый формат wget / curl / browser extensions
- **Сырая строка Header**
  Прямая строка заголовка запроса вида Cookie: SID=...; HSID=...; SSID=...

⊠ Автоматическое распознавание формата при вставке в веб-панели

КРИПТОГРАФИЧЕСКАЯ ЗАЩИТА

ШИФРОВАНИЕ КУК НА ДИСКЕ

- **Симметричный шифр Fernet**
  Стандарт AES-128-CBC с аутентификацией сообщений HMAC-SHA256
- **Права доступа 0600**
  Ключ генерируется в ~/.zv67/key.bin и доступен только текущему пользователю
- **Безопасность репозитория**
  Файл базы credentials.db и ключи намертво исключены из Git через .gitignore

⊠ Расшифровка происходит исключительно в RAM при запуске сессии CDP

РАЗДЕЛЬНЫЕ СРЕДЫ CHROME

ПОЛНАЯ ИЗОЛЯЦИЯ ПРОФИЛЕЙ

- **Отдельные User Data Dir**
  Каждый добавленный аккаунт получает свою изолированную директорию профиля
- **Независимый кэш и хранилище**
  Исключены коллизии авторизации, перемешивание cookie jar и сброс сессий
- **Параллельные сессии**
  Шлюз независимо балансирует входящие запросы между доступными профилями

⊠ Сбои или лимиты одного аккаунта никак не затрагивают остальные

СТРОГО ЛОКАЛЬНЫЙ КОНТУР БЕЗОПАСНОСТИ

Все учётные записи, зашифрованные куки и сокеты управления браузером остаются внутри вашей системы macOS. Никакой телеметрии и утечек в сеть.

Каталог данных: ~/.zv67/ (credentials.db, logs, sessions, key.bin).

МОДЕЛИ И СТАТУС ТЕСТОВ

СТАТУС МОДЕЛЕЙ И ПРОВЕРКА В CODEX

Тестовая матрица, живая работа с Google и нейтрализация ошибок

ZV67 / ZV67-MOCK

100% PASS

Локальный заглушечный провайдер для тестирования конвейера без интернета

- **68 ТЕСТОВ БЭКЕНДА ПРОЙДЕНО**
  Полный набор pytest tests/ подтверждает корректность роутинга и БД
- **СКВОЗНАЯ ИНТЕГРАЦИЯ С CODEX**
  Проверена связка Codex Desktop ⊠ OpenCodeX ⊠ ZV67 FastAPI
- **МГНОВЕННЫЙ ЭХО-ОТВЕТ**
  Идеален для диагностики портов, демона launchd и проверки доступности

GOOGLE-AI-MODE

LIVE: РАБОТАЕТ

Основная модель Google AI Mode через живую сессию Chrome CDP

- **АВТОРИЗАЦИЯ ПО КУКАМ ПРИНЯТА**
  Успешно проверены форматы Cookie-Editor JSON и Netscape cookies.txt
- **ПОЛНЫЙ ДИАЛОГ В ИНТЕРФЕЙСЕ CODEX**
  Запрос успешно отправлен, ответ принят и отображён в окне чата
- **ФОНОВАЯ РАБОТА БЕЗ ОКОН**
  Chrome работает в --headless=new и не перебивает фокус в системе

3.8 FLASH & 4 ARGON

ГОТОВО В UI / 429

Селектор целевых моделей Google AI Mode через веб-интерфейс

- **ПЕРЕКЛЮЧАТЕЛЬ МОДЕЛЕЙ В АДМИНКЕ**
  Реализован выбор моделей и маппинг внутренних параметров Google
- **ПРОБРОС В КАТАЛОГ OPENCODEX**
  Модели автоматически появляются в общем списке моделей Codex
- **ОГРАНИЧЕНИЕ КВОТЫ GOOGLE (429)**
  На тестовых аккаунтах веб-интерфейс Google возвращает квоту 429

КРИТИЧЕСКОЕ ИСПРАВЛЕНИЕ ДЛЯ CODEX

НЕЙТРАЛИЗАЦИЯ ОШИБКИ TOOL CALLING (400)

ПРОБЛЕМА

Codex Desktop автоматически добавляет в каждый запрос системные tools.

При передаче такого запроса модель выдавала фатальную ошибку 400:

```
{"error": {"message": "tool calling is not implemented for this model"}}
```

В результате работа с моделью полностью блокировалась в интерфейсе Codex.

РЕШЕНИЕ ZV67

Шлюз ZV67 перехватывает входящий массив tools и автоматически нормализует тело запроса перед отправкой в веб-интерфейс Google AI Mode.

Codex получает чистый потоковый текстовый ответ без ошибок 400.

- ⊠ Полная прозрачность и стабильность вызова из десктопного приложения
- ⊠ Совместимо со всеми версиями Codex Desktop и OpenCodeX

СИСТЕМНАЯ ИНТЕГРАЦИЯ

ЧИСТЫЙ HEADLESS И СЕРВИС LAUNCHD

Фоновое исполнение процессов и автоматическое управление в macOS

РЕЖИМ БРАУЗЕРА

CHROME --HEADLESS=NEW

- **Никаких всплывающих окон**
  Браузер запускается в современном нативном headless-режиме Chromium
- **Фокус ввода не сбивается**
  Вы можете спокойно писать код и печатать в терминале без прерываний
- **Управление по сокету CDP**
  Шлюз управляет сессией через Chrome DevTools Protocol в фоне

```
google-chrome --headless=new --remote-debugging-port=0 --disable-gpu
```

СИСТЕМНЫЙ ДЕМОН

ДЕМОН MACOS LAUNCHD

- **Автозапуск при старте системы**
  Служба com.zv67.gateway запускается при входе пользователя
- **KeepAlive восстановление**
  Система автоматически поднимает шлюз при аварийном завершении
- **Ротация логов в ~/.zv67/logs**
  Консольный вывод и ошибки пишутся в выделенные файлы журналов

```
launchctl load ~/Library/LaunchAgents/com.zv67.gateway.plist
```

СИНХРОНИЗАЦИЯ МОДЕЛЕЙ

АВТОМАТИЧЕСКИЙ OCX SYNC

- **Интеграция с OpenCodeX**
  Команда scripts/zv67 ocx-sync прописывает провайдер в конфиг OCX
- **Модели сразу в меню Codex**
  Все зарегистрированные модели zv67/* мгновенно появляются в списке
- **Мягкая перезагрузка демона**
  Отправка сигнала SIGHUP серверу OpenCodeX без прерывания сессий

```
./scripts/zv67 ocx-sync && killall -HUP opencode-server
```

ТИХАЯ ФОНОВАЯ СЛУЖБА

После однократной настройки проект не требует внимания.

Шлюз стабильно обрабатывает запросы, пока вы работаете в кодовой базе.

Управление процессами осуществляется через одну утилиту ./scripts/zv67.

РАЗВЁРТЫВАНИЕ

БЫСТРЫЙ СТАРТ И КОМАНДЫ CLI

Пошаговая установка на Mac и единый инструмент управления проектом

ПОШАГОВЫЙ ПЛАН УСТАНОВКИ

ШАГ 1

КЛОНИРОВАНИЕ И ЗАПУСК ИНСТАЛЛЯТОРА

```
git clone <repo-url> && cd zv67 && ./scripts/install.sh
```

Скрипт проверяет зависимости (uv, Node 24, Chrome), создаёт venv и настраивает launchd.

ШАГ 2

ДОБАВЛЕНИЕ УЧЁТНЫХ ДАННЫХ В ВЕБ-ПАНЕЛИ

```
open http://localhost:18080
```

Вставьте экспортированные куки (Cookie-Editor JSON / Netscape) и нажмите «Сохранить».

ШАГ 3

ВЫБОР МОДЕЛИ В ИНТЕРФЕЙСЕ CODEX

```
Модель: zv67/google-ai-mode  (или zv67/zv67-mock для теста)
```

Откройте меню моделей Codex, выберите zv67/google-ai-mode и начинайте диалог.
Шлюз автоматически передаст запрос в AI Mode и вернёт потоковый ответ.

ЕДИНАЯ УТИЛИТА УПРАВЛЕНИЯ SCRIPTS/ZV67

`./scripts/zv67 status`	Отображает статус шлюза, порт 18080, launchd и процессы Chrome
`./scripts/zv67 doctor`	Полная диагностика: зависимости, права 0600, порты и сокеты CDP
`./scripts/zv67 restart`	Перезапускает фоновый демон шлюза через системный launchctl
`./scripts/zv67 logs -f`	Выводит журнал запросов и событий шлюза в режиме реального времени
`./scripts/zv67 ocx-sync`	Синхронизирует модели с OpenCodeX и обновляет каталог моделей

```
Утилита доступна из корня проекта: ./scripts/zv67 [команда]
```

ПРАВИЛА ДОСТУПА

ПРИВАТНЫЙ РЕПОЗИТОРИЙ GITHUB

- **ДОСТУП СТРОГО ПО ПРИГЛАШЕНИЮ**
  Репозиторий закрыт от публичного просмотра. Доступ выдается индивидуально.
- **ИЗОЛЯЦИЯ УЧЁТНЫХ ДАННЫХ**
  Куки и токены хранятся строго локально на машине каждого пользователя.
- **БЕЗОПАСНОСТЬ СЕРВЕРОВ**
  Никакие общие сервера или третьи лица не имеют доступа к вашим сессиям.

```
https://github.com/zverror/zv67-google-ai-mode-codex-bridge
```